

Interrupts

Lecture 10

Josh Brake

Harvey Mudd College

Outline

- Polling, and why it misses events
- The ARM Cortex-M4 exception model
 - The vector table
 - The Nested Vectored Interrupt Controller (NVIC)
- From a pin to your handler: SYSCFG, EXTI, NVIC, and the CPU
- Activity: configure a button interrupt
- Debrief: latency and switch bounce

Learning Objectives

By the end of this lecture you should be able to...

- Explain why polling can miss a short event and how an interrupt catches it.
- Describe what the Cortex-M4 does between an interrupt request and the first line of your handler.
- Trace a GPIO edge through SYSCFG, EXTI, and the NVIC to its handler, and find the register bit that opens each stage.
- Configure an interrupt for any GPIO pin using the reference manual.

Polling

Try It: `button_polling.c`

```
1 while(1){
2     prev_button_state = cur_button_state;
3     cur_button_state = digitalRead(BUTTON_PIN);
4     if ((prev_button_state == 1) &&
5         (cur_button_state == 0)) {
6         led_state = !led_state;
7         digitalWrite(LED_PIN, led_state);
8     }
9     delay_millis(DELAY_TIM, 200);
10 }
```

Groups of 2–3, one board each. Grab a worksheet and clone [HMC-E155/tutorial-interrupts](#).

Part 0: flash `button_polling.c` and tap the button quickly several times. Why are some taps missed?

Interrupt Servicing Sequence

Instead of asking the button over and over, let the hardware tell the CPU when it is pressed.

1. Peripheral _____ interrupt request
2. Processor _____ currently operating task
3. Processor executes an _____ to service the peripheral and optionally clear the interrupt request
4. Processor _____ previously suspended task.

ARM Cortex-M4 Exception Model

Sources of Exceptions

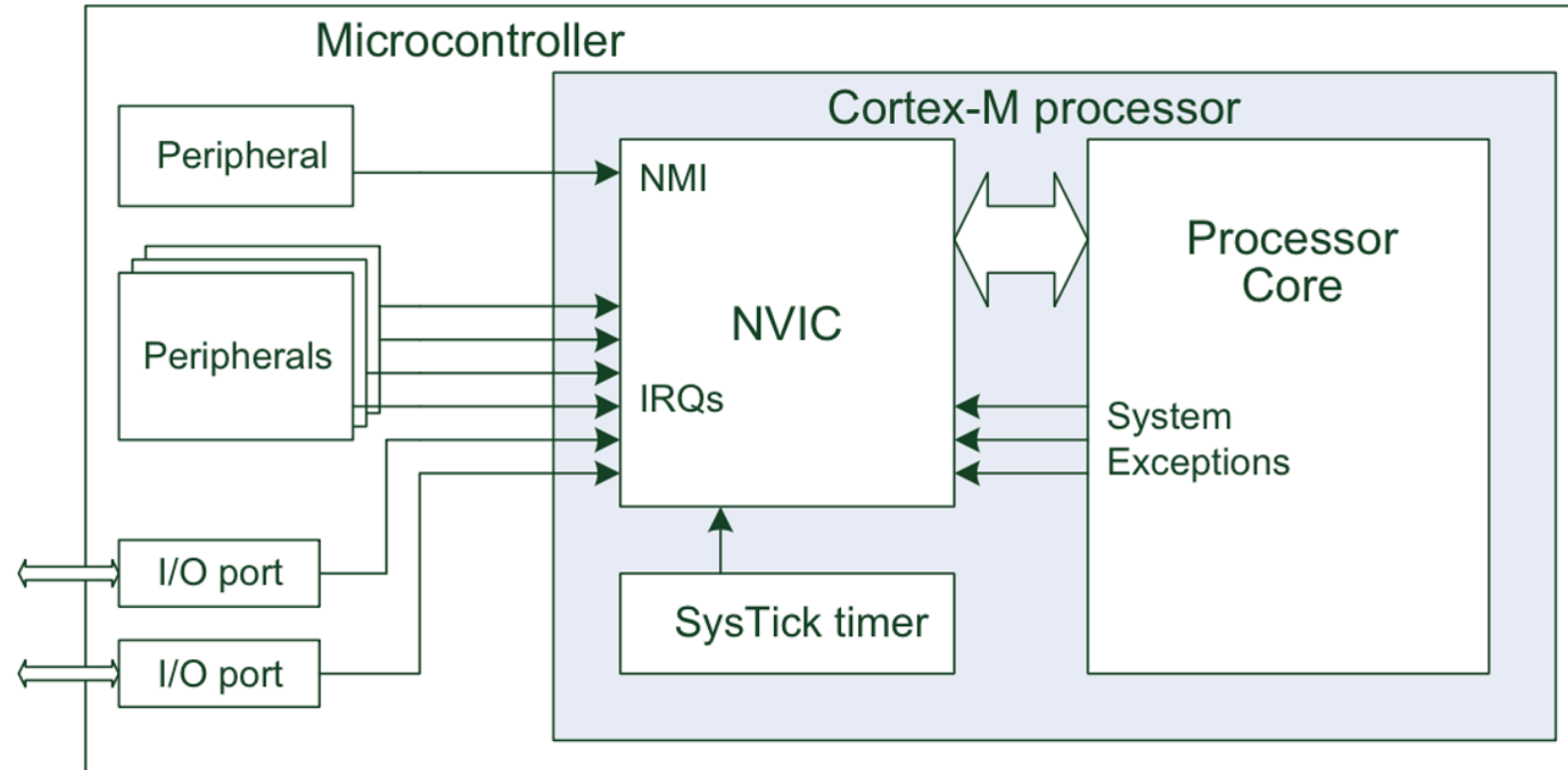


FIGURE 7.1

Various sources of exceptions in a typical microcontroller

Figure 7.1, p.230 The Definitive guide to ARM Cortex-M3 and Cortex-M4 Processors

Vector Table

- Table of addresses to _____ that are placed starting at address **0x00000000**.
- Each memory location contains the _____ of different exception/interrupt handlers
- Each interrupt has a fixed _____ in the table. You will need it to enable the interrupt in the NVIC.

Table 46. STM32L41xxx/42xxx/43xxx/44xxx/45xxx/46xxx vector table (continued)

Position	Priority	Type of priority	Acronym	Description	Address
11	18	settable	DMA1_CH1	DMA1 channel 1 interrupt	0x0000 006C
12	19	settable	DMA1_CH2	DMA1 channel 2 interrupt	0x0000 0070
13	20	settable	DMA1_CH3	DMA1 channel 3 interrupt	0x0000 0074
14	21	settable	DMA1_CH4	DMA1 channel 4 interrupt	0x0000 0078
15	22	settable	DMA1_CH5	DMA1 channel 5 interrupt	0x0000 007C
16	23	settable	DMA1_CH6	DMA1 channel 6 interrupt	0x0000 0080
17	24	settable	DMA1_CH7	DMA1 channel 7 interrupt	0x0000 0084
18	25	settable	ADC1_2	ADC1 and ADC2 ⁽²⁾ global interrupt	0x0000 0088
19	26	settable	CAN1_TX ⁽¹⁾	CAN1_TX interrupts	0x0000 008C
20	27	settable	CAN1_RX0 ⁽¹⁾	CAN1_RX0 interrupts	0x0000 0090
21	28	settable	CAN1_RX1 ⁽¹⁾	CAN1_RX1 interrupt	0x0000 0094
22	29	settable	CAN1_SCE ⁽¹⁾	CAN1_SCE interrupt	0x0000 0098
23	30	settable	EXTI9_5	EXTI Line[9:5] interrupts	0x0000 009C
24	31	settable	TIM1_BRK/TIM15	TIM1 Break/TIM15 global interrupts	0x0000 00A0
25	32	settable	TIM1_UP/TIM16	TIM1 Update/TIM16 global interrupts	0x0000 00A4
26	33	settable	TIM1_TRG_COM	TIM1 trigger and commutation interrupt	0x0000 00A8
27	34	settable	TIM1_CC	TIM1 capture compare interrupt	0x0000 00AC
28	35	settable	TIM2	TIM2 global interrupt	0x0000 00B0
29	36	settable	TIM3 ⁽³⁾	TIM3 global interrupt	0x0000 00B4
30	37	settable	-	Reserved	0x0000 00B8
31	38	settable	I2C1_EV	I2C1 event interrupt	0x0000 00BC
32	39	settable	I2C1_ER	I2C1 error interrupt	0x0000 00C0
33	40	settable	I2C2_EV ⁽⁴⁾	I2C2 event interrupt	0x0000 00C4
34	41	settable	I2C2_ER ⁽⁴⁾	I2C2 error interrupt	0x0000 00C8
35	42	settable	SPI1	SPI1 global interrupt	0x0000 00CC
36	43	settable	SPI2 ⁽⁴⁾	SPI2 global interrupt	0x0000 00D0
37	44	settable	USART1	USART1 global interrupt	0x0000 00D4
38	45	settable	USART2	USART2 global interrupt	0x0000 00D8
39	46	settable	USART3 ⁽⁴⁾	USART3 global interrupt	0x0000 00DC
40	47	settable	EXTI15_10	EXTI Line[15:10] interrupts	0x0000 00E0
41	48	settable	RTC_ALARM	RTC alarms through EXTI line 18 interrupts	0x0000 00E4
42	49	settable	-	Reserved	0x0000 00E8
43	50	settable	-	Reserved	0x0000 00EC

The Vector Table in Code

stm32l432xx_Vectors.s

```
1  _vectors:
2      VECTOR __stack_end__
3      VECTOR Reset_Handler
4      EXC_HANDLER NMI_Handler
5      VECTOR HardFault_Handler
6      ...
7      EXC_HANDLER SysTick_Handler
8      //
9      // External interrupts
10     //
11     ISR_HANDLER WWDG_IRQHandler
12     ISR_HANDLER PVD_PVM_IRQHandler
13     ...
```

- Entry 0 is the initial _____, and entry 1 is the initial PC, _____.
- External interrupts start after the 16 system exceptions, so **WWDG** is position 0.
- To handle an interrupt, write a C function with **exactly** the name in the table. The linker puts its address in that entry.

Nested Vectored Interrupt Controller (NVIC)

- On Cortex-M4 the NVIC supports up to 240 IRQs, a Non-Maskable Interrupt, a SysTick timer interrupt, and a number of system exceptions.
- : each interrupt has its own entry in the vector table, so the CPU jumps straight to the right handler.
- : each interrupt has a priority, so a higher-priority interrupt can preempt a lower-priority handler that is already running.
 - A lower number means a higher priority. Reset, NMI, and HardFault have fixed negative priorities and preempt everything.
- The NVIC is part of the Cortex-M4 core, so its registers are documented in the programming manual (PM0214), not in RM0394.

Handling an Interrupt

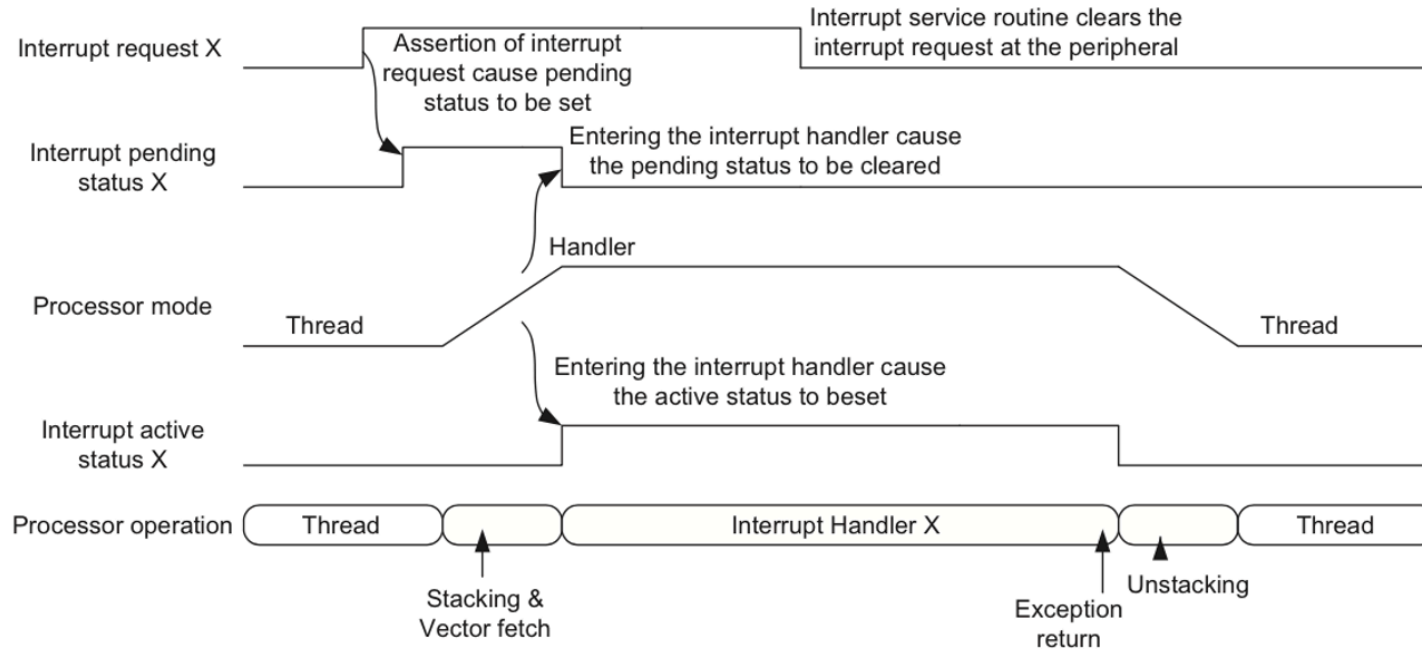


FIGURE 7.14

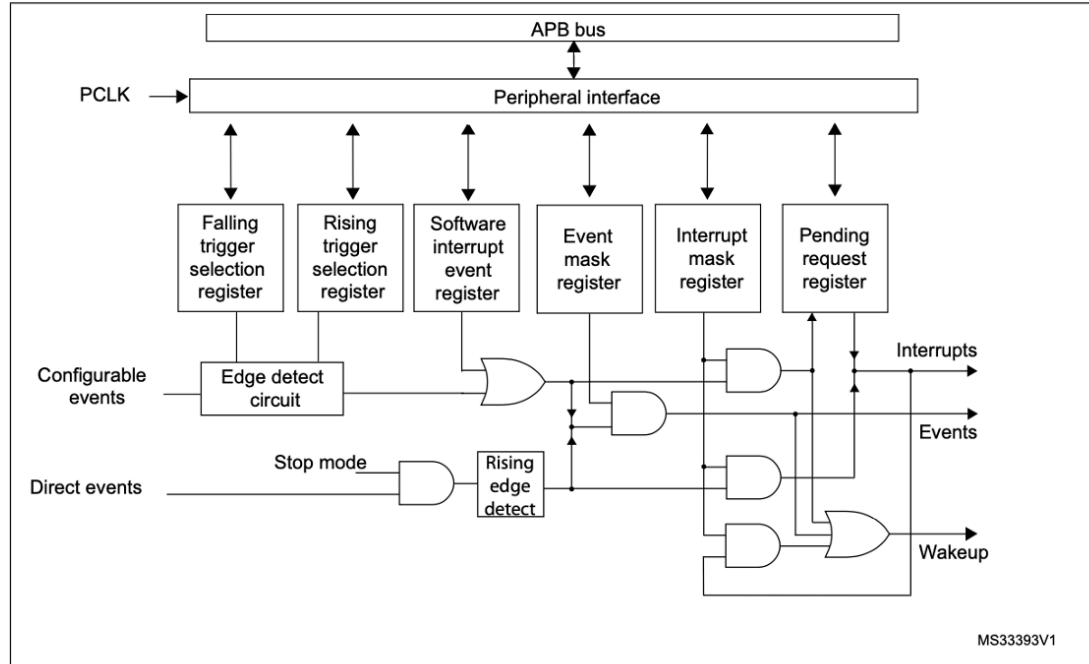
A simple case of interrupt pending and activation behavior

The hardware saves registers on the stack before the handler and restores them after, so the handler is an ordinary _____. It must clear the request at the peripheral before it returns.

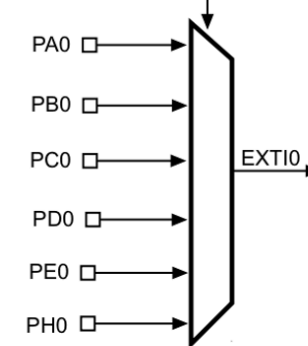
From Pin to Handler

External Interrupt/Event Controller (EXTI)

Figure 28. Configurable interrupt/event block diagram

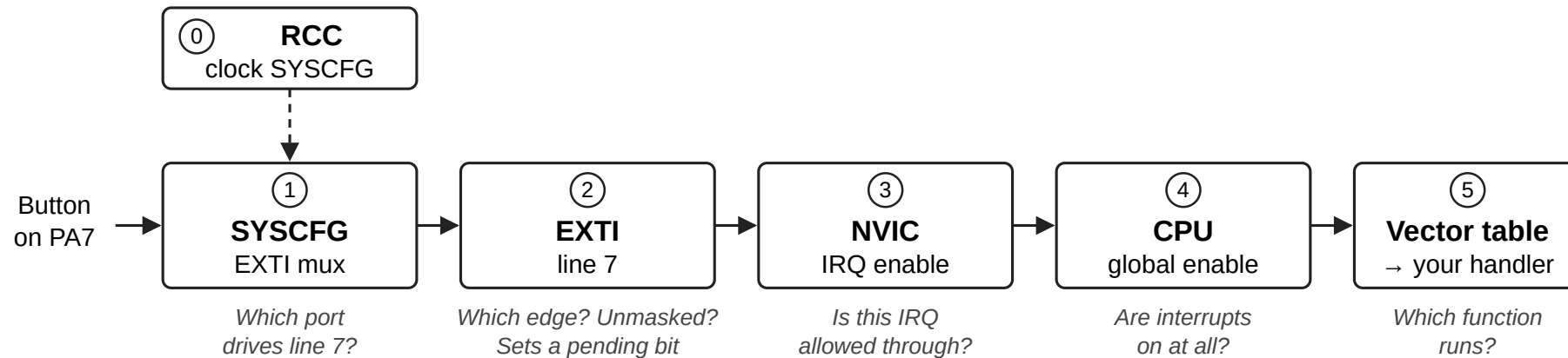


EXTI0[3:0] bits in the SYSCFG_EXTICR1 register



EXTI1[3:0] bits in the SYSCFG_EXTICR1 register

A Press Has to Get Through Every Stage



A press reaches your code only if every stage passes it on. Leave one closed and nothing happens, with no error message.

Each stage is a register bit, and the pin number ($n = 7$) tells you where to find almost all of them.

Activity: Configure a Button Interrupt

Same groups and boards as Part 0.

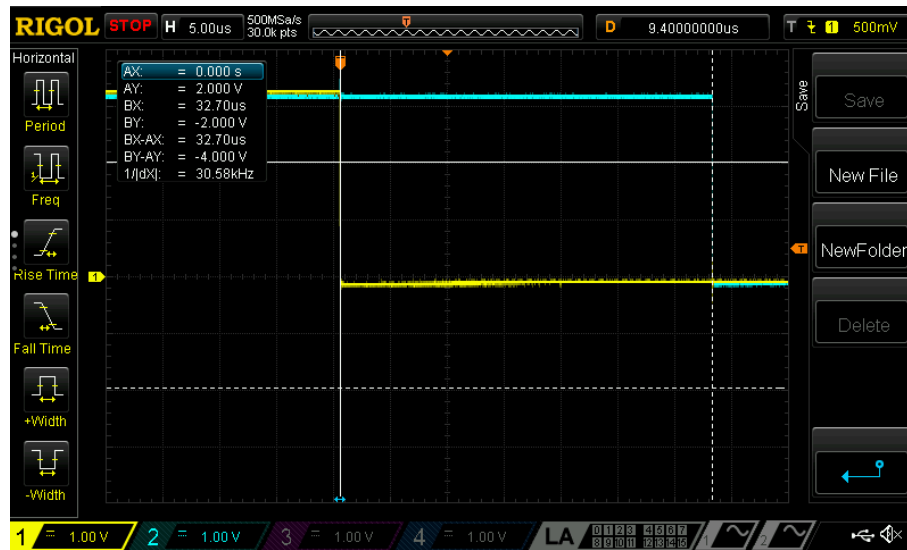
- **Part 1 (15 min):** open each stage for PA7, then fill in the TODOs in `main()` of `button_interrupt.c`.
- **Part 2 (10 min):** name and write the handler. Predict what each mistake does, then try one.
- **Part 3 (10 min):** add the board's second button, on PA4, with no hints.

Reference: RM0394 Ch. 6 (RCC), 9 (SYSCFG), 12 (NVIC and the vector table), 13 (EXTI).
PM0214 §4.3 (NVIC registers).

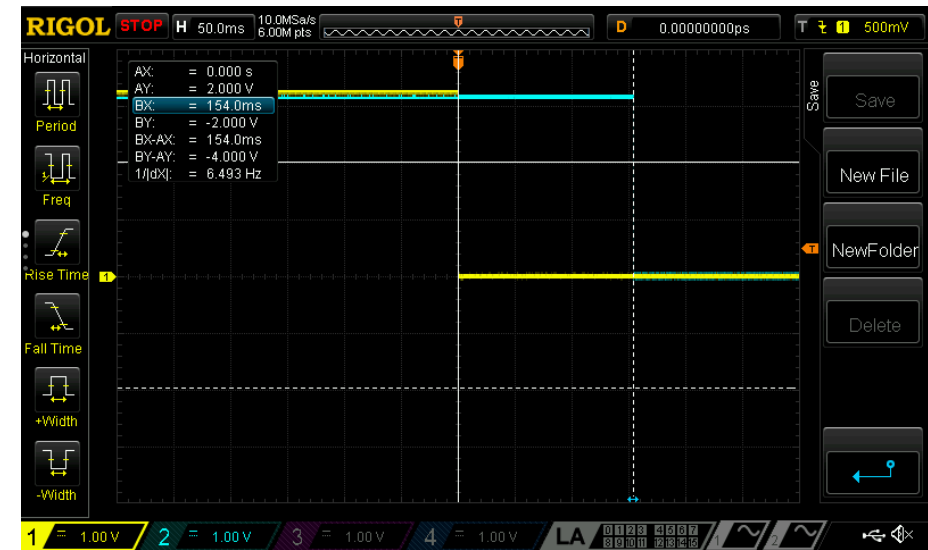
Registers are accessed just like in your Lab 4 drivers: `EXTI->IMR1`, `NVIC->ISER[0]`. The device header `stm32l432xx.h` defines the structs.

Debrief

Latency: Interrupt vs. Polling with 200 ms Delay

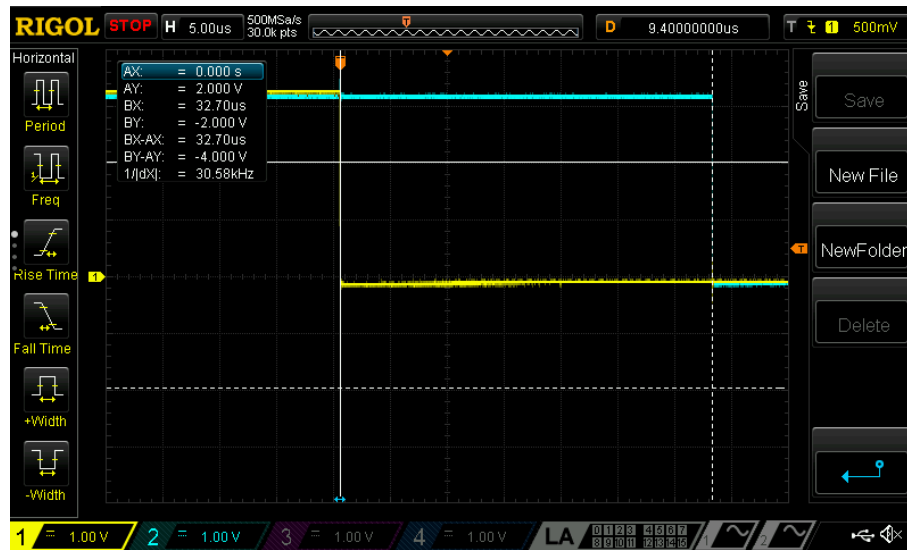


Interrupt

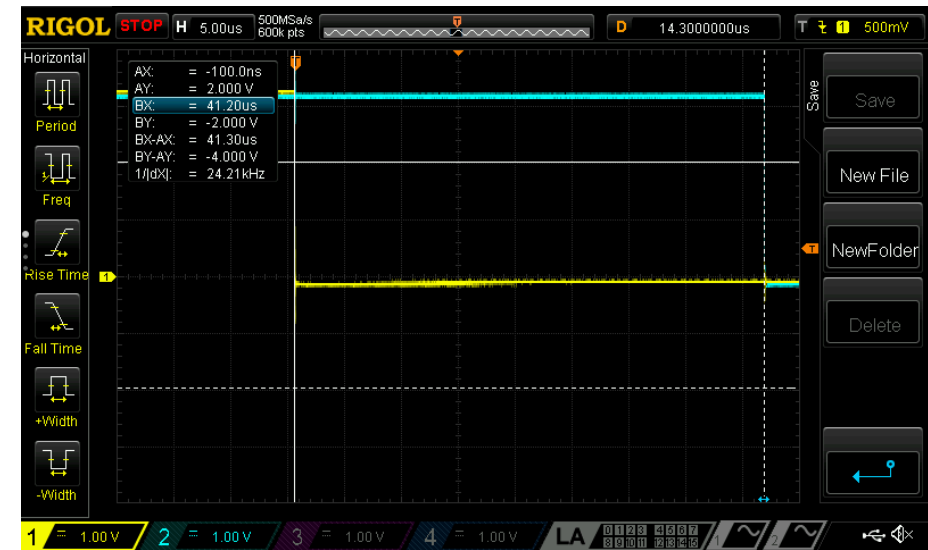


Polling, 200 ms delay

Latency: Interrupt vs. Polling with No Delay



Interrupt



Polling, no delay

Why Does the LED Sometimes Toggle Twice?

- The switch for a few milliseconds on press and release. Each bounce is a new falling edge, so EXTI fires several times per press.
- Polling with a 200 ms delay hides this because it samples so slowly.
- One fix, in `button_debounce.c` on the `solution` branch: drop EXTI and sample the button from a **timer interrupt** (TIM6) every 5 ms. Only accept a new state once 4 samples in a row agree.
 - Same stages, different peripheral: TIM6 sets a flag, the NVIC enables IRQ 54 (`ISER1` bit 22), and `TIM6_DAC_IRQHandler` clears the flag.
 - The cost is up to 20 ms of extra latency.

Wrap Up

- Polling spends CPU time checking and can still miss events. Interrupts let the hardware tell the CPU.
- On an interrupt, the CPU stacks registers, looks up the handler in the vector table, runs it, and resumes.
- A GPIO interrupt passes through SYSCFG, EXTI, the NVIC, and the CPU's global enable. Each stage is a bit you have to set.
- The handler's name must match the vector table, and the handler must clear the pending bit.
- Lab 5 uses the same steps on two encoder pins.